training_morogoro

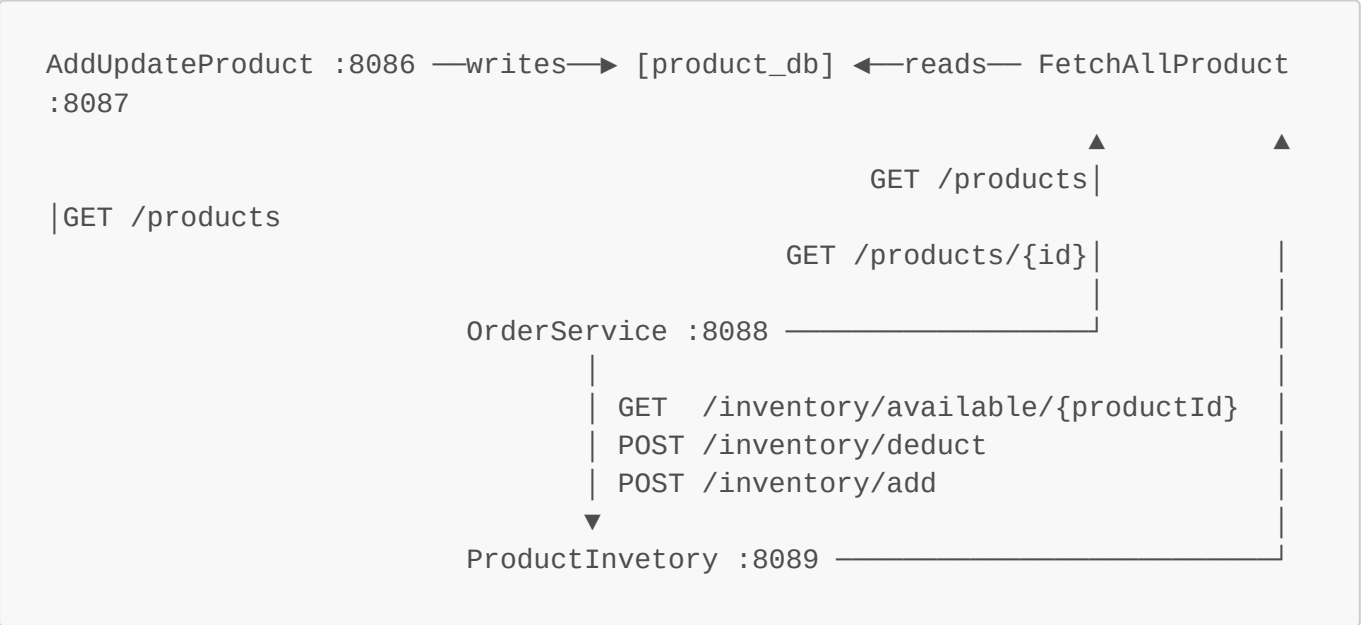
Four Spring Boot services for managing products, inventory and orders. Each service uses Spring Data JPA with MySQL and exposes a REST API that other services and Postman can call. Services call each other with OpenFeign clients (`@FeignClient`).

Services

Service	Port	Database	Responsibility
AddUpdateProduct	8086	product_db	Add and update products, and seed 12 sample products on startup
FetchAllProduct	8087	product_db	Fetch all products, a product by id, or products by name
OrderService	8088	order_db	Place, cancel and list orders and order items
ProductInvetory	8089	inventory_db	Record stock ADD/DEDUCT actions with opening and closing quantities

AddUpdateProduct and FetchAllProduct share the `products` table in `product_db`. A product added on 8086 can be read on 8087.

Communication



All four services can also be called directly from Postman.

Caller	Called service	Endpoint	When
OrderService	FetchAllProduct	`GET /products/{id}`	Placing an order, to get the product price

Caller	Called service	Endpoint	When
OrderService	ProductInvetory	`GET /inventory/available/{productId}`	Placing an order, to check stock
OrderService	ProductInvetory	`POST /inventory/deduct`	Placing an order, to reduce stock
OrderService	ProductInvetory	`POST /inventory/add`	Cancelling an order, to return stock
OrderService	FetchAllProduct	`GET /products`	Listing orders, to show product names on order items
ProductInvetory	FetchAllProduct	`GET /products`	Listing inventory, to show product names

If FetchAllProduct is down, the order and inventory lists still return, but `productName` is `null`.

Place order flow

1. Postman sends `POST localhost:8088/orders` with `{"productId":1,"quantity":2}`.
2. OrderService gets the product from FetchAllProduct. If the product is not found, it returns `404`.
3. OrderService gets the available quantity from ProductInvetory. If the quantity is too low, it returns `400`.
4. OrderService asks ProductInvetory to deduct the quantity. ProductInvetory saves a DEDUCT record.
5. OrderService saves an `Order` with status `PLACED`, and an `OrderItem` with the price and total amount.

Cancel order flow

1. Postman sends `PUT localhost:8088/orders/{id}/cancel`.
2. For each order item, OrderService asks ProductInvetory to add the quantity back. ProductInvetory saves an ADD record.
3. OrderService sets the order status to `CANCELLED`.

API reference

AddUpdateProduct (8086)

Method	Path	Body
POST	`/products`	`{"name":"Laptop","description":"Dell","price":1500000}`
PUT	`/products/{id}`	`{"name":"Laptop","description":"Dell","price":1400000}`

FetchAllProduct (8087)

Method	Path	Returns
GET	`/products`	All products
GET	`/products/{id}`	One product, or `404`
GET	`/products/name/{name}`	Products whose name contains `{name}`, or `404`

OrderService (8088)

Method	Path	Body / Returns
POST	`/orders`	Body `{"productId":1,"quantity":2}`, returns the order
PUT	`/orders/{id}/cancel`	Returns the cancelled order
GET	`/orders`	All orders, each with order items and product names
GET	`/orders/{id}/items`	Items of one order

ProductInvetory (8089)

Method	Path	Body / Returns
POST	`/inventory/add`	Body `{"productId":1,"quantity":50}`
POST	`/inventory/deduct`	Body `{"productId":1,"quantity":5}`, returns `400` if stock is too low
GET	`/inventory`	All records with product names
GET	`/inventory/available/{productId}`	Current available quantity
GET	`/inventory/{productId}`	All records for one product
GET	`/inventory/{productId}/today`	Today's records for one product

Each ADD or DEDUCT saves one record. The opening quantity is the previous closing quantity:

productId	openingQuantity	quantity	closingQuantity	action	actionDate
1	0	50	50	ADD	2026-10-29
1	50	2	48	DEDUCT	2026-10-29

Running

1. Make sure MySQL is running with user `db_user`. The databases are created automatically on first start.
2. Start the services in this order: AddUpdateProduct, FetchAllProduct, ProductInvetory, OrderService. Run each one with `./mvnw spring-boot:run` from its folder.
3. Add stock with `POST localhost:8089/inventory/add`, then place an order with `POST localhost:8088/orders`.